CSE4007
Artificial Intelligence

Lecture 9: Generative Models Part 1

Introduction & Autoregressive models

Department of Data Science
Hanyang University

Why Study Generative Models?

- Simulate possible futures for planning or simulated RL
- Realistic generation tasks
  - E.g., video/image manipulation, cartoon character generation
- Data manipulation
  - E.g., Image enhancement, video frame interpolation, clothing transla tion, etc.
- Anomaly detection
- Etc.

Supervised vs unsupervised learning

	supervised
data	$(\boldsymbol{x}, y)$ $\boldsymbol{x}$: data, y: label
goal	learn a *function* to map $\boldsymbol{x} \mapsto y$
examples	classification regression object detection semantic segmentation

Supervised vs unsupervised learning

	supervised
data	$(\boldsymbol{x}, y)$ $\boldsymbol{x}$: data, y: label
goal	learn a *function* to map $\boldsymbol{x} \mapsto y$
examples	classification regression object detection semantic segmentation

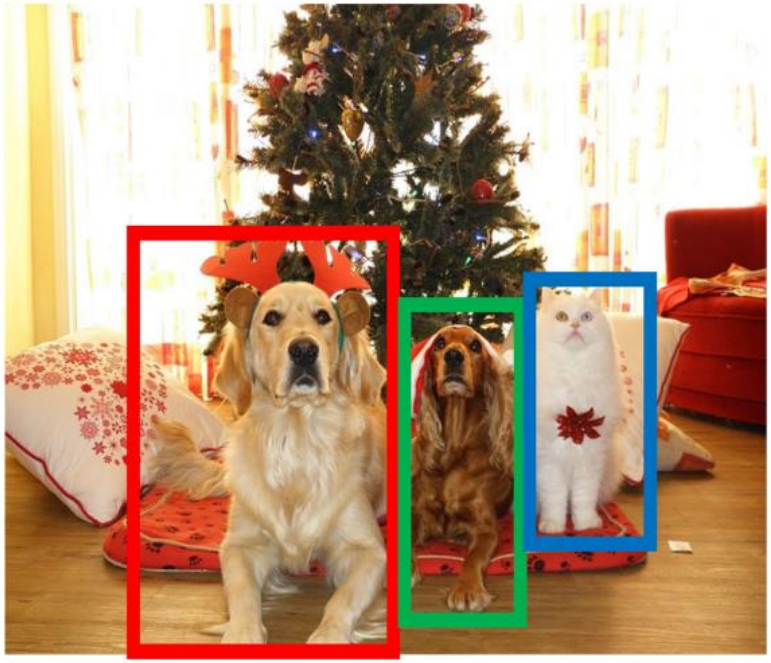
Classification

Cat

Supervised vs unsupervised learning

	supervised
data	$(\boldsymbol{x}, y)$ $\boldsymbol{x}$: data, y: label
goal	learn a *function* to map $\boldsymbol{x} \mapsto y$
examples	classification regression object detection semantic segmentation

Object Detection

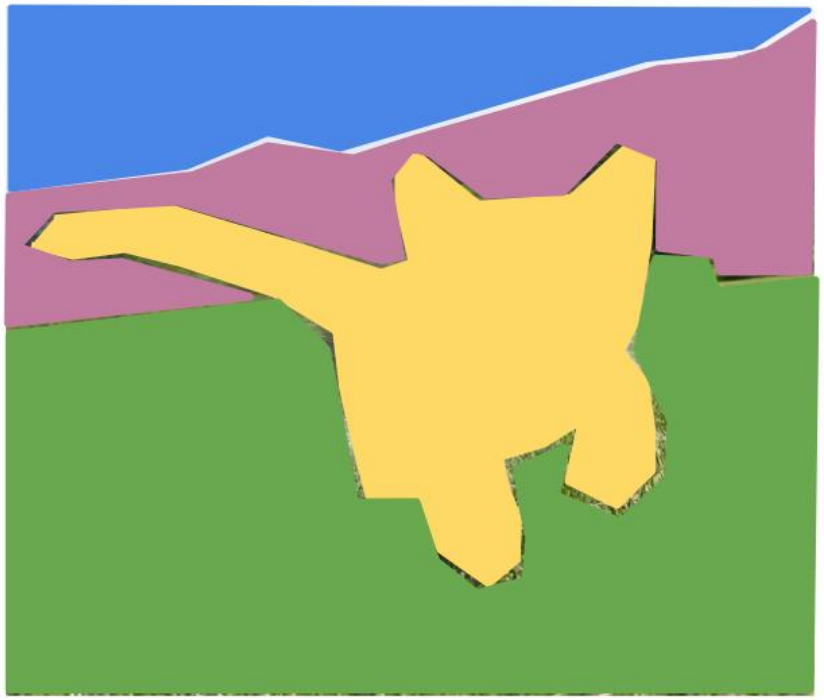


DOG, DOG, CAT

Supervised vs unsupervised learning

	supervised
data	$(\boldsymbol{x}, y)$ $\boldsymbol{x}$: data, y: label
goal	learn a *function* to map $\boldsymbol{x} \mapsto y$
examples	classification regression object detection semantic segmentation

Semantic Segmentation



GRASS, CAT, TREE, SKY

Supervised vs unsupervised learning

	supervised
data	$(\boldsymbol{x}, y)$ $\boldsymbol{x}$: data, y: label
goal	learn a *function* to map $\boldsymbol{x} \mapsto y$
examples	classification regression object detection semantic segmentation

Image captioning

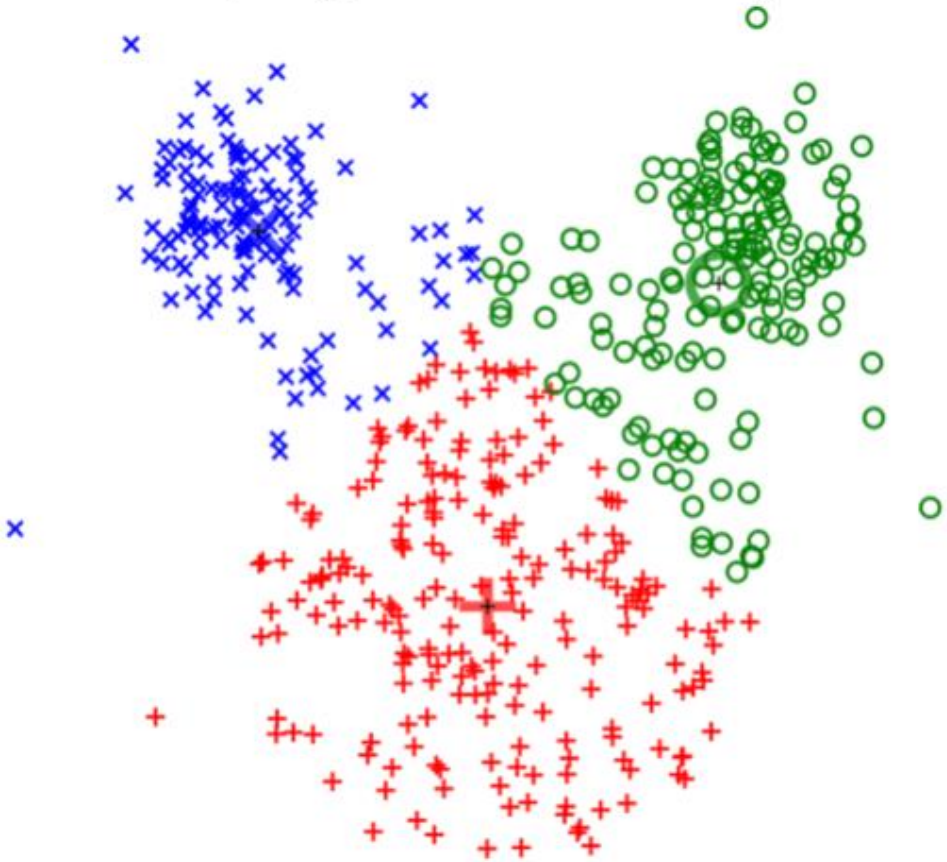
A cat sitting on a suitcase on the floor

Supervised vs unsupervised learning

	supervised	**unsupervised**
data	$(\boldsymbol{x}, y)$ $\boldsymbol{x}$: data, y: label	$\boldsymbol{x}$ just data, no label
goal	learn a *function* to map $\boldsymbol{x} \mapsto y$	learn inherent *structure* of the data
examples	classification regression object detection semantic segmentation	clustering dimensionality reduction representation learning density estimation

Supervised vs unsupervised learning

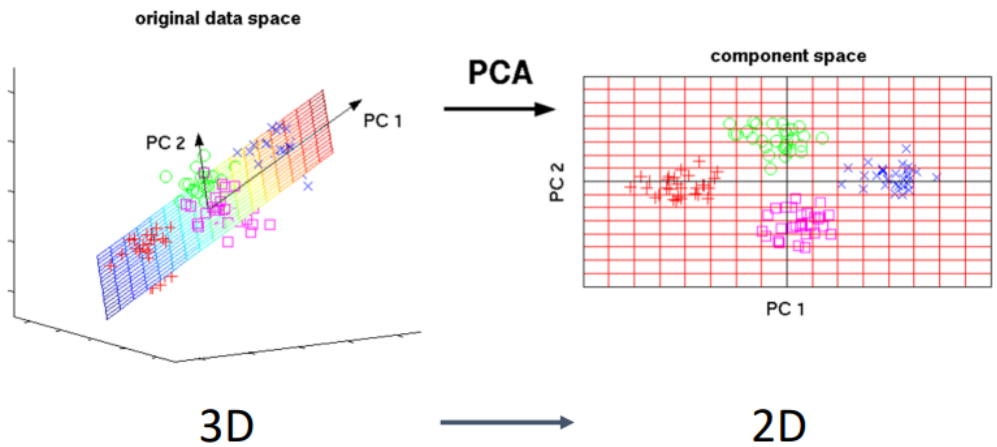
Clustering
(e.g. K-Means)



unsupervised
$\boldsymbol{x}$ just data, no label
learn inherent *structure* of the data
clustering dimensionality reduction representation learning density estimation

Supervised vs unsupervised learning

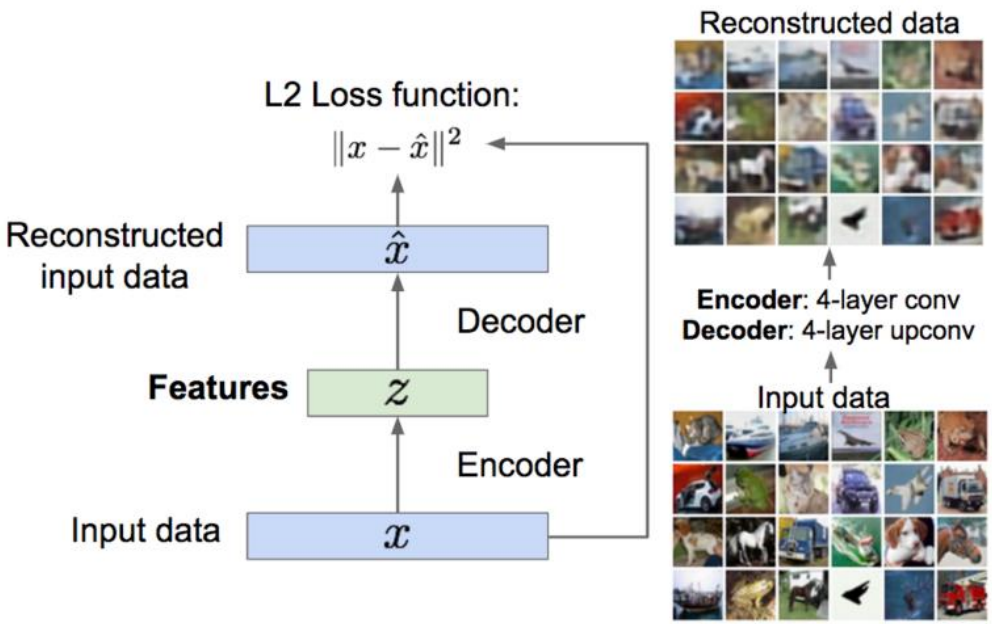
Dimensionality Reduction
(e.g. Principal Components Analysis)



unsupervised
$\boldsymbol{x}$ just data, no label
learn inherent *structure* of the data
clustering dimensionality reduction representation learning density estimation

Supervised vs unsupervised learning

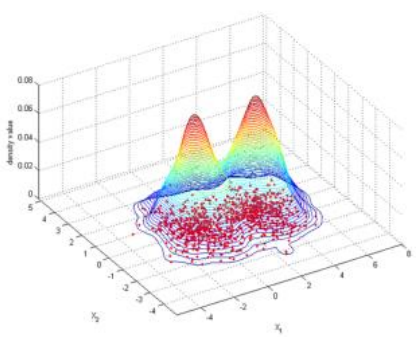
Feature Learning
(e.g. autoencoders)

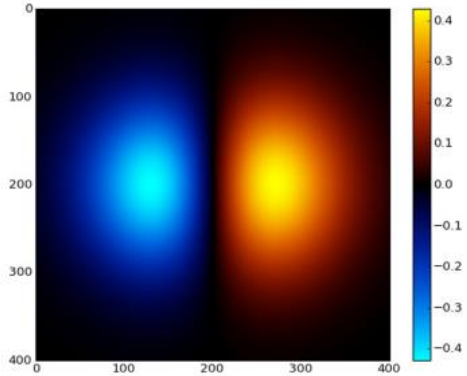


unsupervised
x just data, no label
learn inherent *structure* of the data
clustering dimensionality reduction representation learning density estimation

Supervised vs unsupervised learning

Density Estimation





unsupervised
$\boldsymbol{x}$ just data, no label
learn inherent *structure* of the data
clustering dimensionality reduction representation learning density estimation

Discriminative vs Generative Models

- **Discriminative Model**: Learn a probability distribution p(y|x)

- **Generative Model**: Learn a probability distribution p(x)

- **Conditional Generative Model**: Learn P(x|y)

Data: x

Label: y

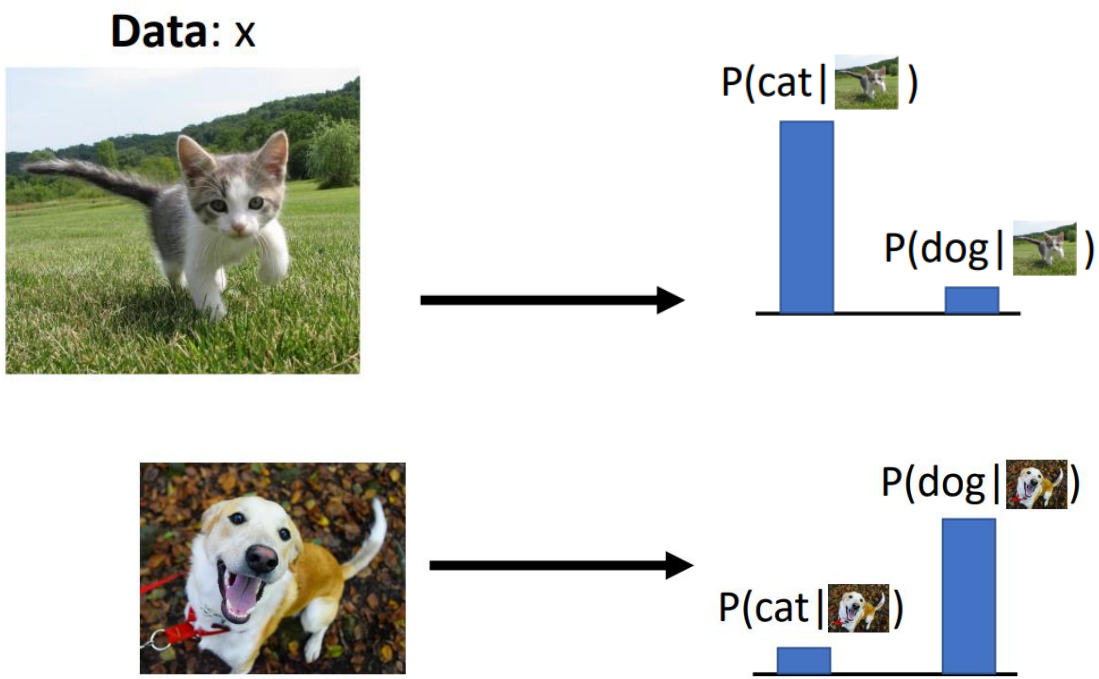
Cat

Discriminative vs Generative Models

- **Discriminative Model**: Learn a probability distribution p(y|x)

- **Generative Model**: Learn a probability distribution p(x)

- **Conditional Generative Model**: Learn P(x|y)



The possible labels for each input "compete" for probability mass.

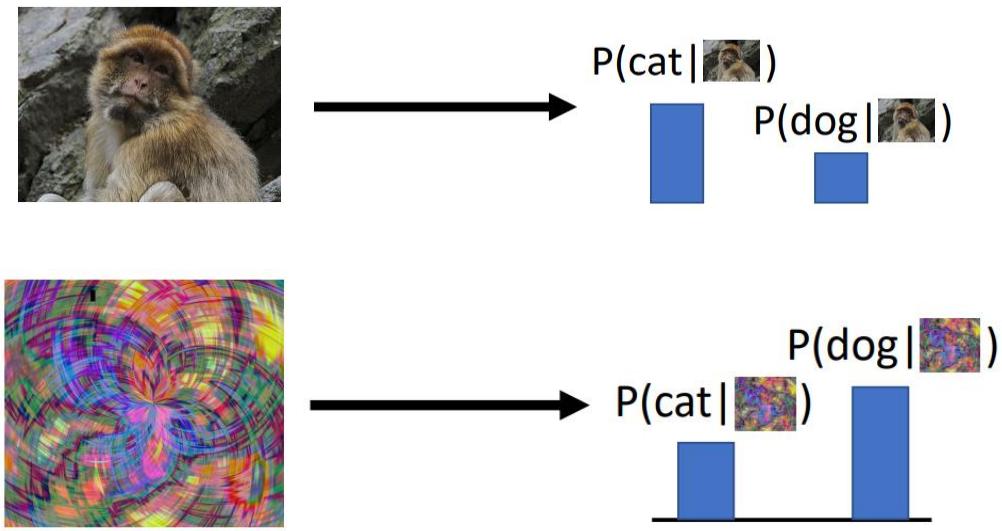
But no competition between **images**

Discriminative vs Generative Models

- **Discriminative Model**: Learn a probability distribution p(y|x)

- **Generative Model**: Learn a probability distribution p(x)

- **Conditional Generative Model**: Learn P(x|y)



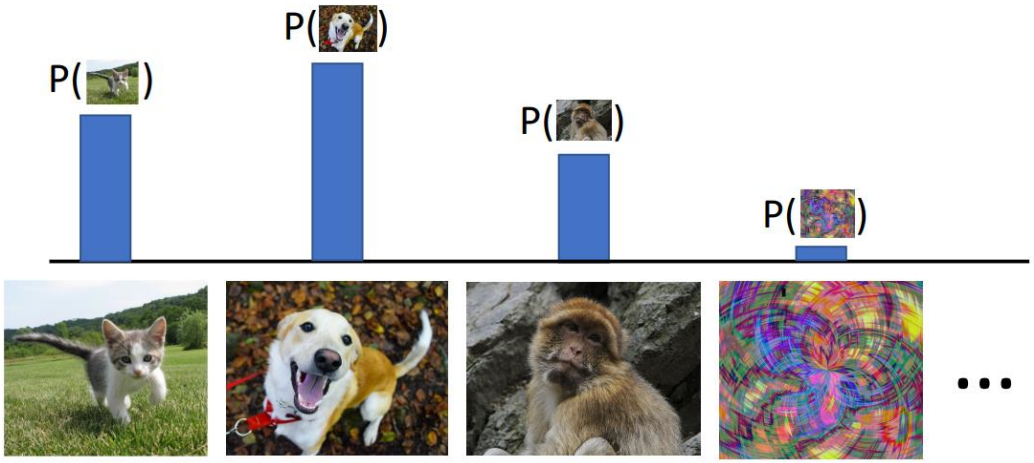
No way for the model to handle unreasonable inputs; it must give label distribution for all images

Discriminative vs Generative Models

- **Discriminative Model**: Learn a probability distribution p(y|x)

- **Generative Model**: Learn a probability distribution p(x)

- **Conditional Generative Model**: Learn P(x|y)



All possible images compete with each other for probability mass

Requires deep image understanding! Is a dog more likely to sit or stand? How about 3-legged dog vs 3-armed monkey?

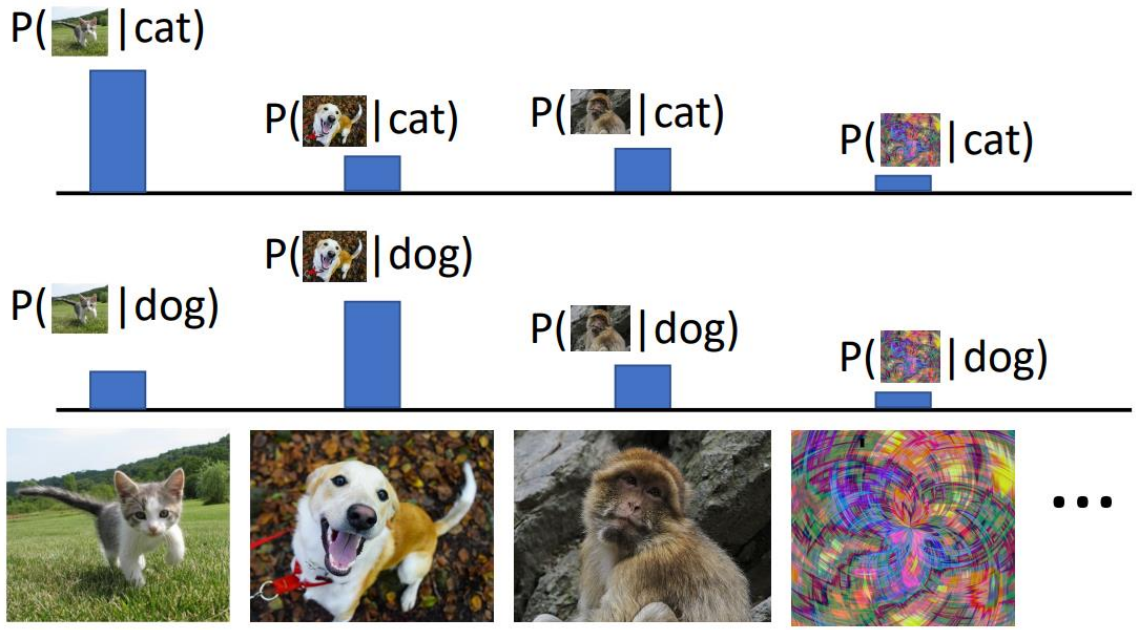
Model can “reject” unreasonable inputs by assigning them small values

Discriminative vs Generative Models

- **Discriminative Model**: Learn a probability distribution p(y|x)

- **Generative Model**: Learn a probability distribution p(x)

- **Conditional Generative Model**: Learn P(x|y)



Each possible label induces a competition among all images

Discriminative vs Generative Models

- **Discriminative Model**: Learn a probability distribution p(y|x)

- **Generative Model**: Learn a probability distribution p(x)

- **Conditional Generative Model**: Learn P(x|y)

Bayes' Rule:

$$P(x \mid y) = \frac{P(y \mid x)}{P(y)} P(x)$$

Discriminative vs Generative Models

- **Discriminative Model**: Learn a probability distribution p(y|x)

- **Generative Model**: Learn a probability distribution p(x)

- **Conditional Generative Model**: Learn P(x|y)

Bayes' Rule:

$$\underbrace{\boxed{P(x \mid y)}}_{\text{Conditional Generative Model}} = \frac{\overbrace{\boxed{P(y \mid x)}}^{\text{Discriminative Model}}}{\underbrace{\boxed{P(y)}}_{\text{Prior over labels}}} \overbrace{\boxed{P(x)}}^{\text{(Unconditional) Generative Model}}$$

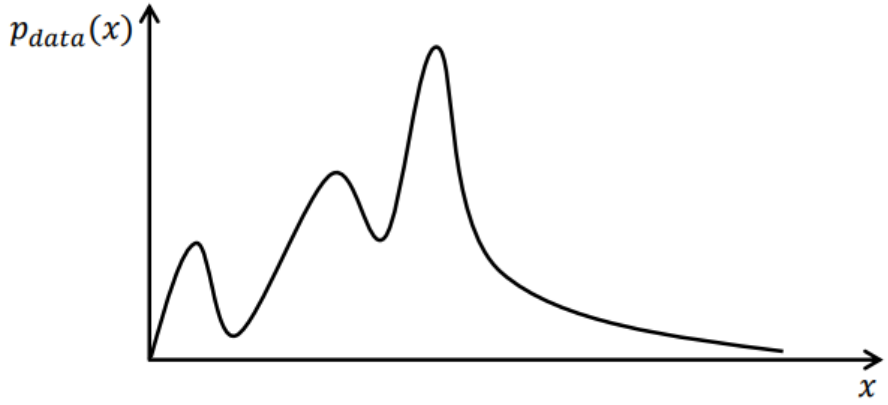
We can build a conditional generative model from other components!

What can we do with these models?

- **Discriminative Model**: Learn a probability distribution p(y|x) → Assign labels to data; Feature learning (with labels)
- **Generative Model**: Learn a probability distribution p(x) → Detect outliers; Feature learning (without labels); Sample to generate new data
- **Conditional Generative Model**: Learn P(x|y) → Assign labels, while rejecting outliers!; Generate new data conditioned on input labels

Generative Model

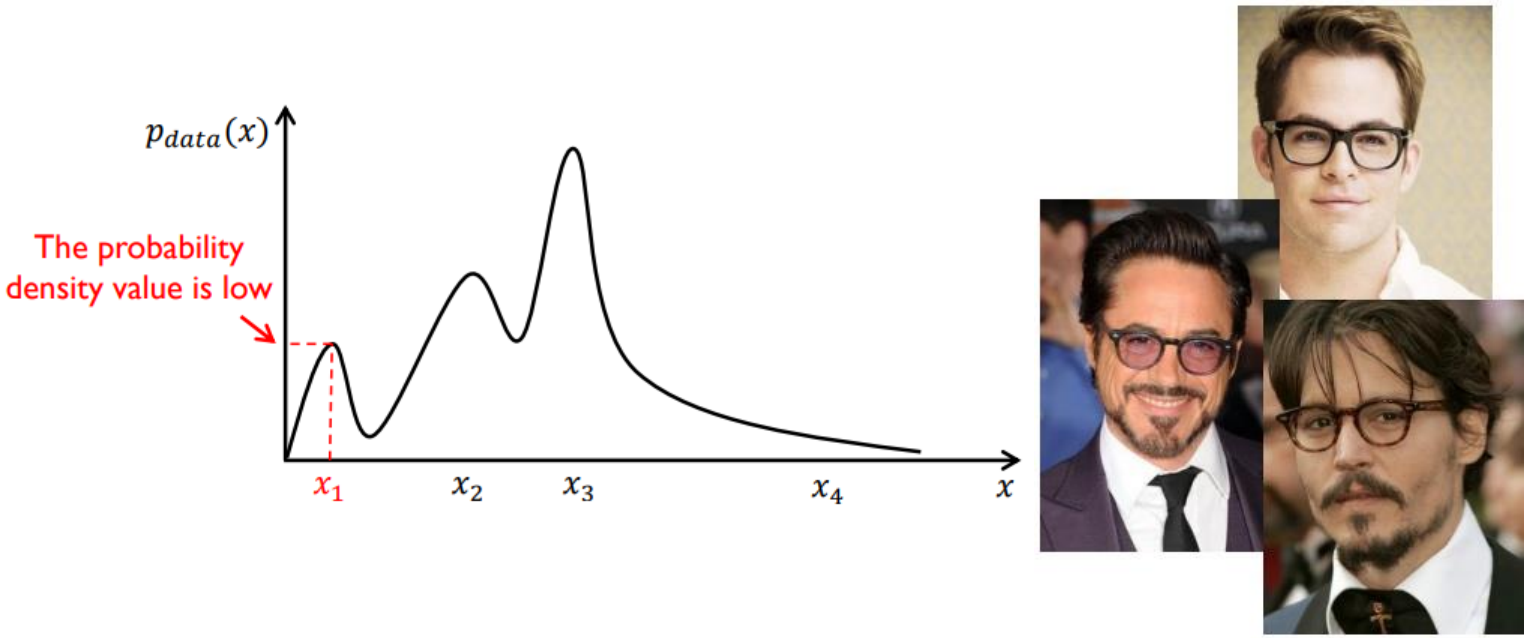
- There is a $p_{data}(x)$ that represents the distribution of actual images
- Example: dataset of human face images



Slide modified from 'Generative Adversarial Networks' by Yunjey Choi: https://www.slideshare.net/NaverEngineering/1-gangenerative-adversarial-network

Generative Model

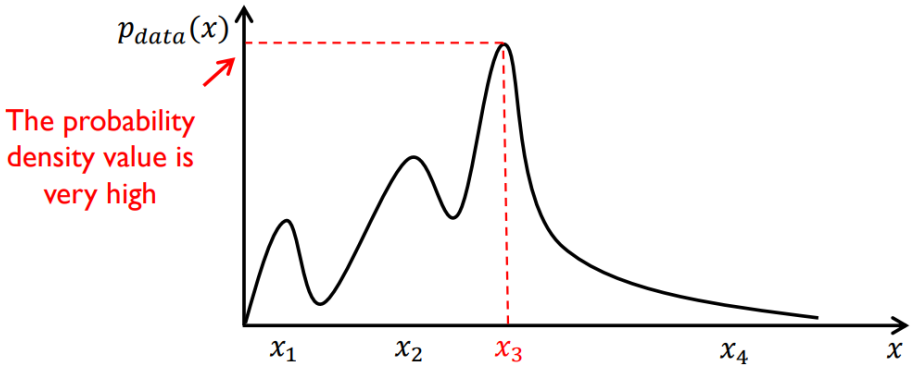
- The dataset may contain few images of men with glasses



Slide modified from 'Generative Adversarial Networks' by Yunjey Choi: https://www.slideshare.net/NaverEngineering/1-gangenerative-adversarial-network

Generative Model

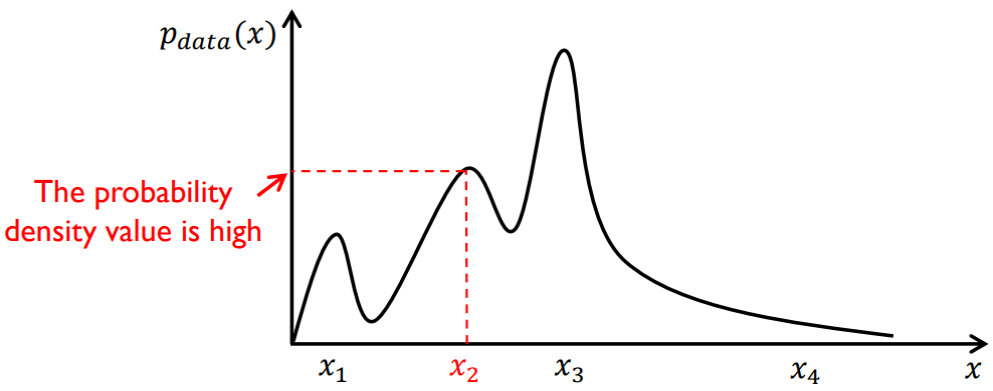
- The dataset may contain very many images of women with blonde hair



Slide modified from 'Generative Adversarial Networks' by Yunjey Choi: https://www.slideshare.net/NaverEngineering/1-gangenerative-adversarial-network

Generative Model

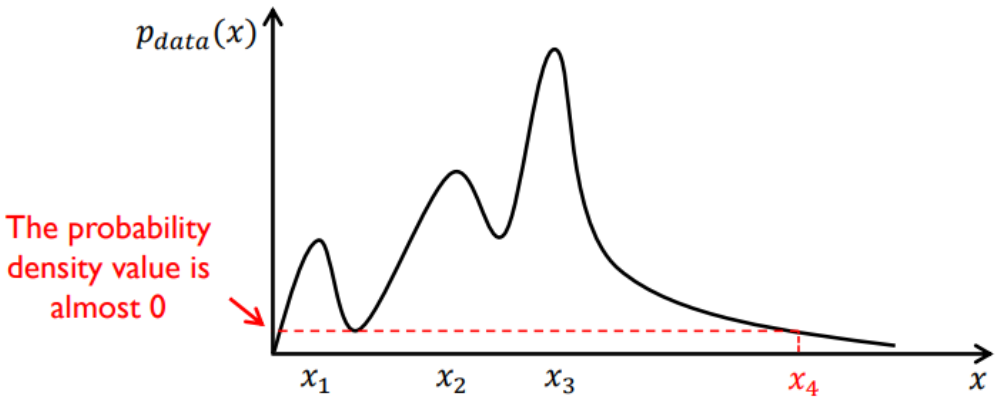
- The dataset may contain very many images of women with blonde hair



Slide modified from 'Generative Adversarial Networks' by Yunjey Choi: https://www.slideshare.net/NaverEngineering/1-gangenerative-adversarial-network

Generative Model

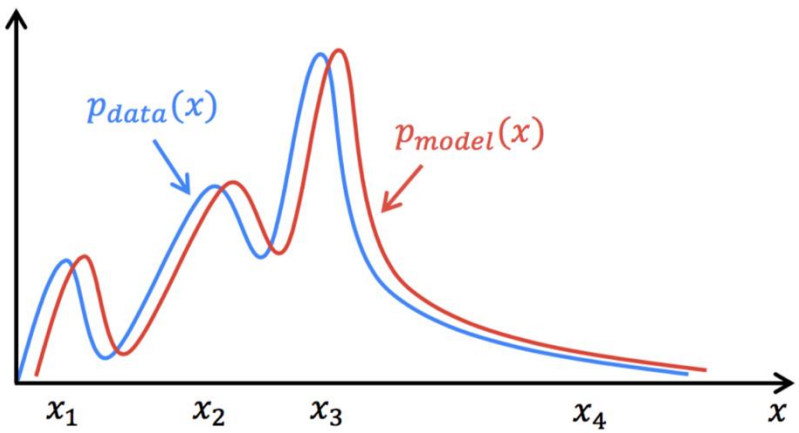
- The dataset may not contain these strange images



Slide modified from 'Generative Adversarial Networks' by Yunjey Choi: https://www.slideshare.net/NaverEngineering/1-gangenerative-adversarial-network

Generative Model

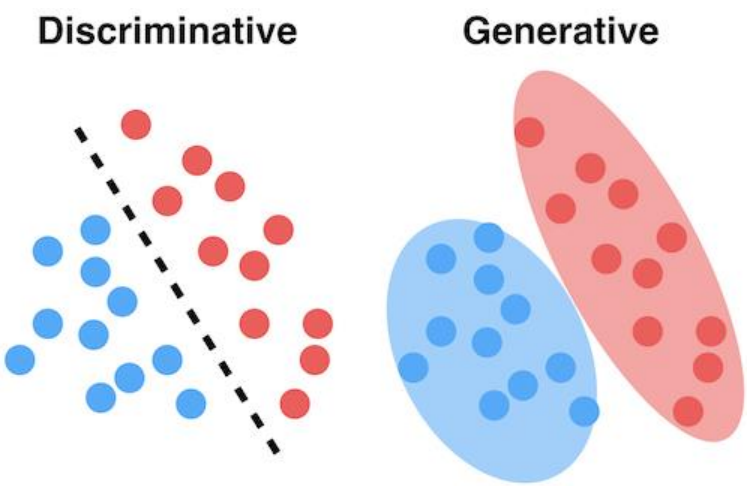
- The goal of the generative model is to find a $p_{model}(x)$ that a pproximates $p_{data}(x)$ well.



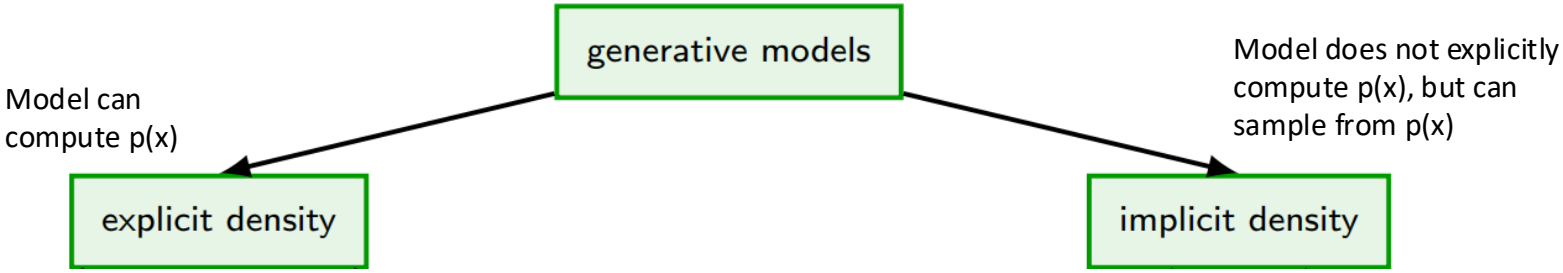
Generative vs Discriminative

- Training generative models is difficult

- Generative models have to model *more* (*whole* data distribut ion).

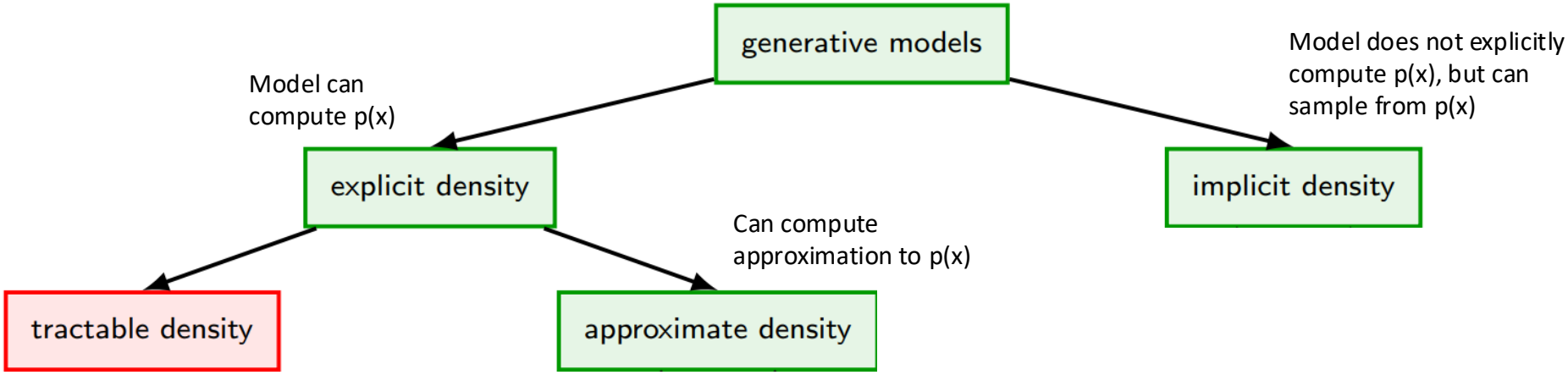


Taxonomy of generative models



Ian Goodfellow, Tutorial on Generative Adversarial Networks, 2017

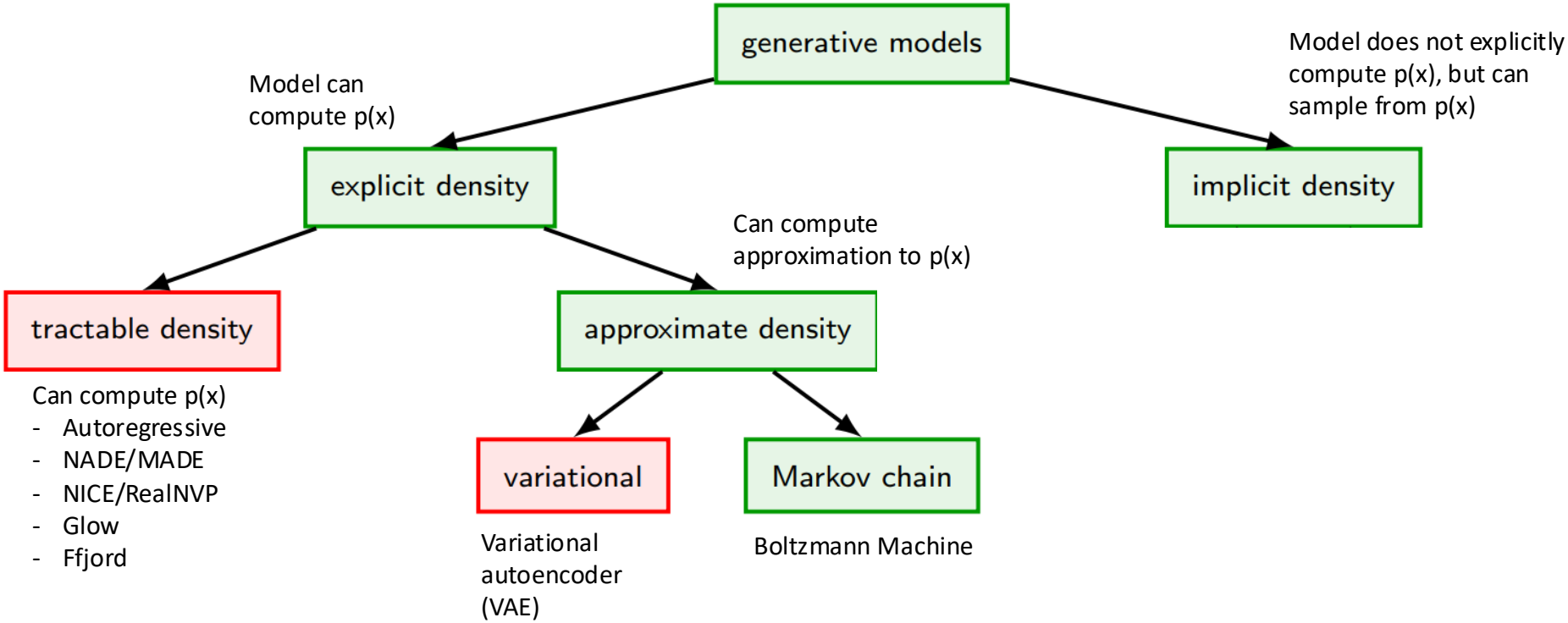
Taxonomy of generative models



Can compute p(x)
- Autoregressive
- NADE/MADE
- NICE/RealNVP
- Glow
- Ffjord

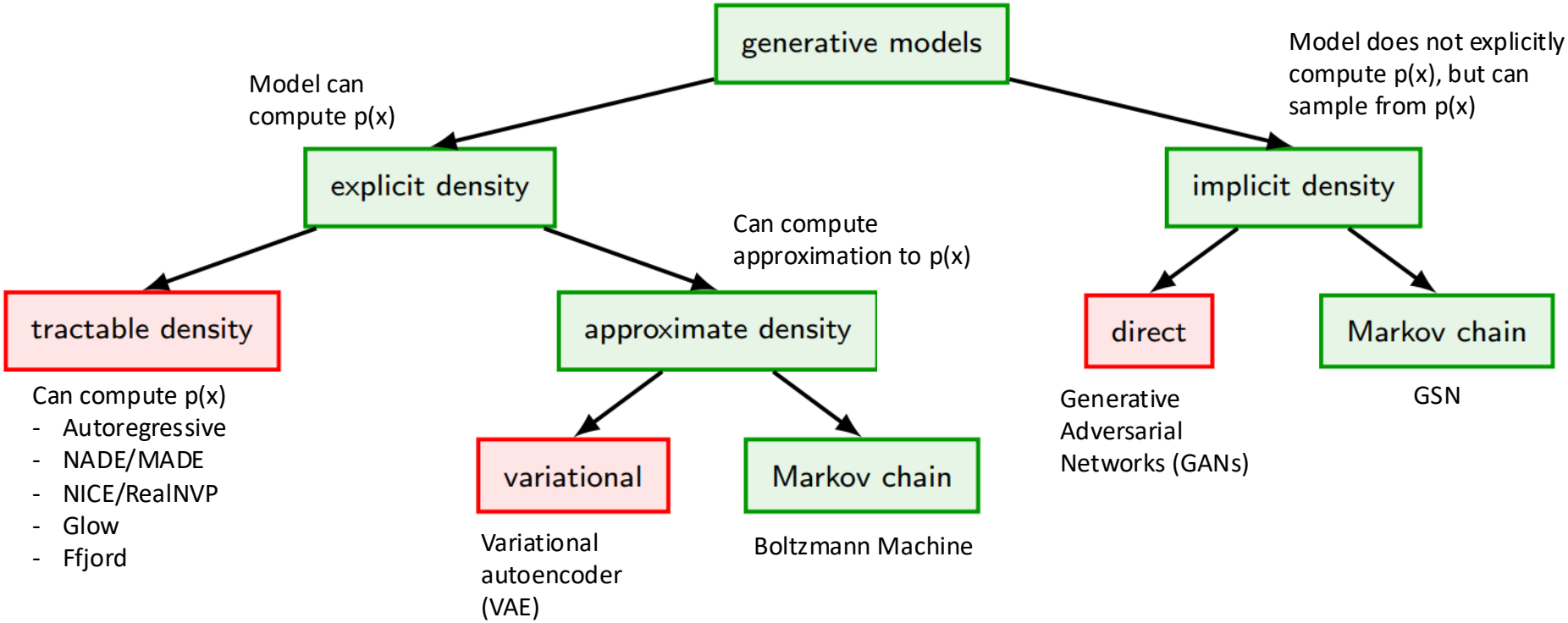
Ian Goodfellow, Tutorial on Generative Adversarial Networks, 2017

Taxonomy of generative models



Ian Goodfellow, Tutorial on Generative Adversarial Networks, 2017

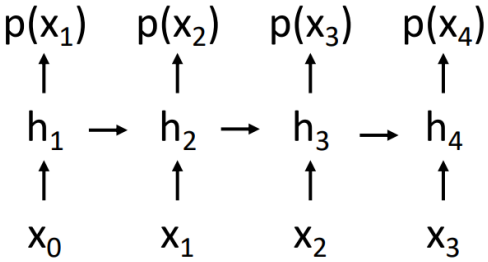
Taxonomy of generative models



Ian Goodfellow, Tutorial on Generative Adversarial Networks, 2017

Autoregressive Models

Fully visible belief network (FVBN)



We've already seen this!
Language modeling with an RNN!

- Explicit density model
  - Model $p(\boldsymbol{x})$ as a learnable function $f(\boldsymbol{x}, \boldsymbol{w})$
- Assumption
  - $\boldsymbol{x}$ consists of multiple subparts $\boldsymbol{x} = (x_1, x_2, x_3, ..., x_n)$
- How it works
  - Decompose the likelihood of each input $\boldsymbol{x}$ into a product of 1D distributions using chain rule

$$\underbrace{p(\boldsymbol{x})}_{\substack{\text{likelihood}\\ \text{of input } \boldsymbol{x}}} = \underbrace{\prod_{i=1}^{n} p(x_i \mid x_1, \ldots, x_{i-1})}_{\substack{\text{probability of } i\text{-th feature}\\ \text{given all previous features}}}$$

  - Maximize the likelihood of training data

$$\boldsymbol{w}^* = \arg\max_{\boldsymbol{w}} \prod_i p(\boldsymbol{x}^{(i)})$$

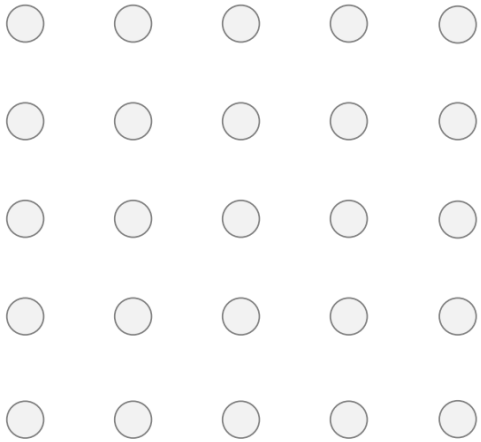
PixelRNN

- Generate image pixels one at a time, starting at the upper left corner

- Compute a hidden state for each pixel that depends on hidden states and RGB values from the left and from above (LSTM recurrence)

$$\boldsymbol{h}_{x,y} = f(\boldsymbol{h}_{x-1,y}, \boldsymbol{h}_{x,y-1}, \boldsymbol{w})$$

- At each pixel, predict red, then blue, then green (i.e., RGB): softmax over [0, 1, …, 255]



Van den Oord et al, "Pixel Recurrent Neural Networks", ICML 2016

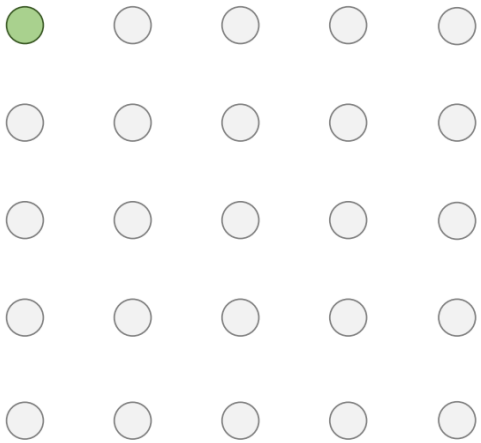
PixelRNN

- Generate image pixels one at a time, starting at the upper left corner

- Compute a hidden state for each pixel that depends on hidden states and RGB values from the left and from above (LSTM recurrence)

$$\boldsymbol{h}_{x,y} = f(\boldsymbol{h}_{x-1,y}, \boldsymbol{h}_{x,y-1}, \boldsymbol{w})$$

- At each pixel, predict red, then blue, then green (i.e., RGB): softmax over [0, 1, …, 255]



Van den Oord et al, "Pixel Recurrent Neural Networks", ICML 2016

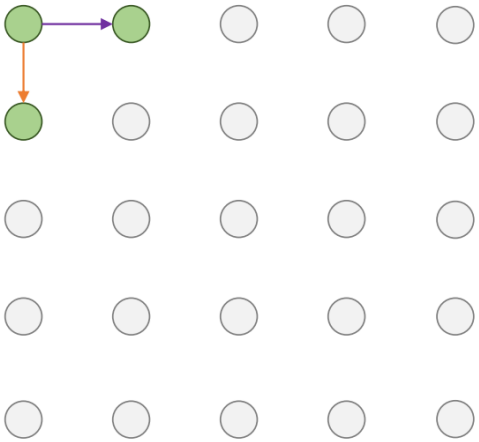
PixelRNN

- Generate image pixels one at a time, starting at the upper left corner

- Compute a hidden state for each pixel that depends on hidden states and RGB values from the left and from above (LSTM recurrence)

$$\boldsymbol{h}_{x,y} = f(\boldsymbol{h}_{x-1,y}, \boldsymbol{h}_{x,y-1}, \boldsymbol{w})$$

- At each pixel, predict red, then blue, then green (i.e., RGB): softmax over [0, 1, …, 255]



Van den Oord et al, "Pixel Recurrent Neural Networks", ICML 2016

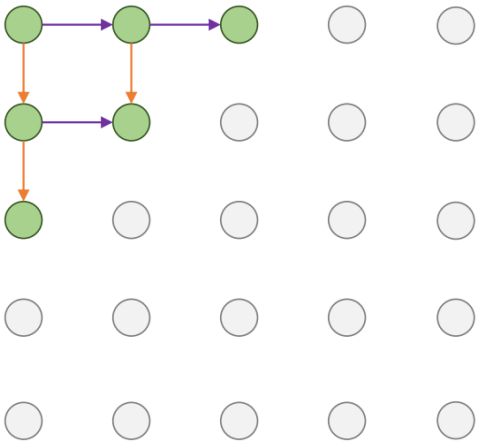
PixelRNN

- Generate image pixels one at a time, starting at the upper left corner

- Compute a hidden state for each pixel that depends on hidden states and RGB values from the left and from above (LSTM recurrence)

$$\boldsymbol{h}_{x,y} = f(\boldsymbol{h}_{x-1,y}, \boldsymbol{h}_{x,y-1}, \boldsymbol{w})$$

- At each pixel, predict red, then blue, then green (i.e., RGB): softmax over [0, 1, …, 255]



Van den Oord et al, "Pixel Recurrent Neural Networks", ICML 2016

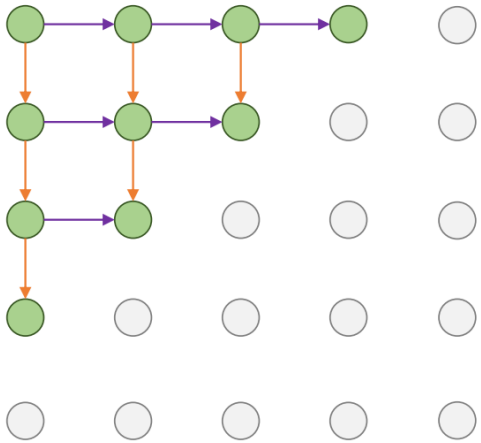
PixelRNN

- Generate image pixels one at a time, starting at the upper left corner

- Compute a hidden state for each pixel that depends on hidden states and RGB values from the left and from above (LSTM recurrence)

$$\boldsymbol{h}_{x,y} = f(\boldsymbol{h}_{x-1,y}, \boldsymbol{h}_{x,y-1}, \boldsymbol{w})$$

- At each pixel, predict red, then blue, then green (i.e., RGB): softmax over [0, 1, …, 255]



Van den Oord et al, "Pixel Recurrent Neural Networks", ICML 2016

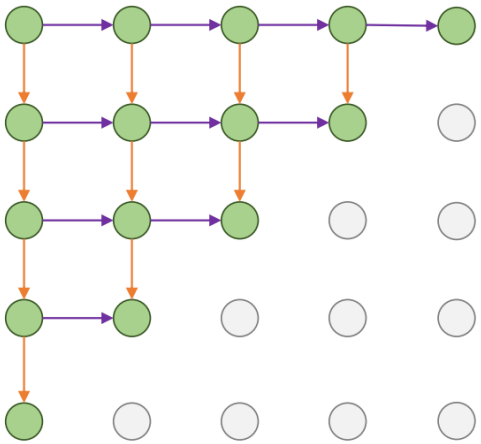
PixelRNN

- Generate image pixels one at a time, starting at the upper left corner

- Compute a hidden state for each pixel that depends on hidden states and RGB values from the left and from above (LSTM recurrence)

$$\boldsymbol{h}_{x,y} = f(\boldsymbol{h}_{x-1,y}, \boldsymbol{h}_{x,y-1}, \boldsymbol{w})$$

- At each pixel, predict red, then blue, then green (i.e., RGB): softmax over [0, 1, …, 255]



Van den Oord et al, "Pixel Recurrent Neural Networks", ICML 2016

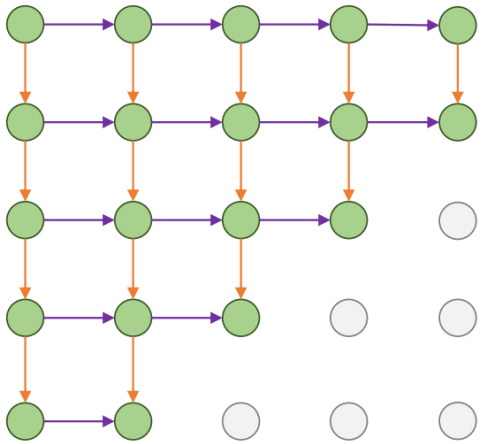
PixelRNN

- Generate image pixels one at a time, starting at the upper left corner

- Compute a hidden state for each pixel that depends on hidden states and RGB values from the left and from above (LSTM recurrence)

$$\boldsymbol{h}_{x,y} = f(\boldsymbol{h}_{x-1,y}, \boldsymbol{h}_{x,y-1}, \boldsymbol{w})$$

- At each pixel, predict red, then blue, then green (i.e., RGB): softmax over [0, 1, …, 255]



Van den Oord et al, "Pixel Recurrent Neural Networks", ICML 2016

PixelRNN

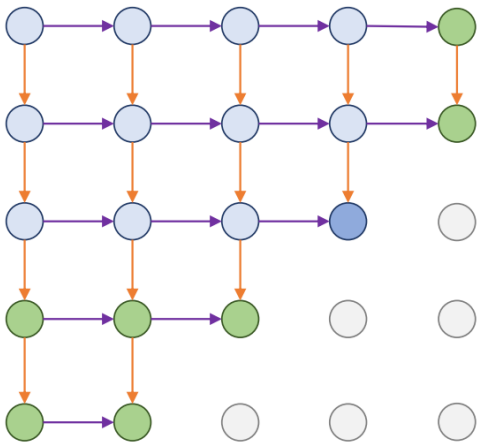
- Generate image pixels one at a time, starting at the upper left corner

- Compute a hidden state for each pixel that depends on hidden states and RGB values from the left and from above (LSTM recurrence)

$$\boldsymbol{h}_{x,y} = f(\boldsymbol{h}_{x-1,y}, \boldsymbol{h}_{x,y-1}, \boldsymbol{w})$$

- At each pixel, predict red, then blue, then green (i.e., RGB): softmax over [0, 1, …, 255]

- Each pixel depends **implicitly** on all pixels above and to the left



Van den Oord et al, "Pixel Recurrent Neural Networks", ICML 2016

PixelRNN

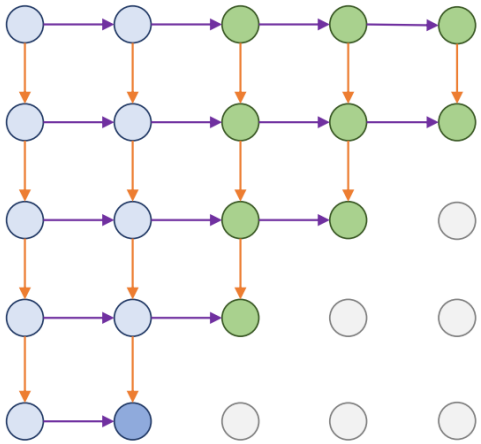
- Generate image pixels one at a time, starting at the upper left corner

- Compute a hidden state for each pixel that depends on hidden states and RGB values from the left and from above (LSTM recurrence)

$$\boldsymbol{h}_{x,y} = f(\boldsymbol{h}_{x-1,y}, \boldsymbol{h}_{x,y-1}, \boldsymbol{w})$$

- At each pixel, predict red, then blue, then green (i.e., RGB): softmax over [0, 1, …, 255]

- Each pixel depends **implicitly** on all pixels above and to the left



Van den Oord et al, "Pixel Recurrent Neural Networks", ICML 2016

PixelRNN

- Generate image pixels one at a time, starting at the upper left corner

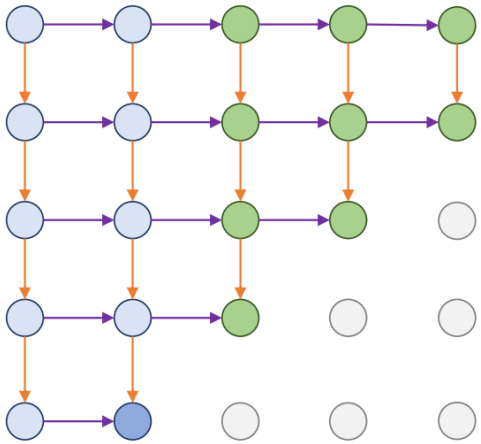
- Compute a hidden state for each pixel that depends on hidden states and RGB values from the left and from above (LSTM recurrence)

$$\boldsymbol{h}_{x,y} = f(\boldsymbol{h}_{x-1,y}, \boldsymbol{h}_{x,y-1}, \boldsymbol{w})$$

- At each pixel, predict red, then blue, then green (i.e., RGB): softmax over [0, 1, …, 255]

- Each pixel depends **implicitly** on all pixels above and to the left

Problem: Very slow during both training and testing; N x N image requires 2N – 1 sequential steps



Van den Oord et al, "Pixel Recurrent Neural Networks", ICML 2016

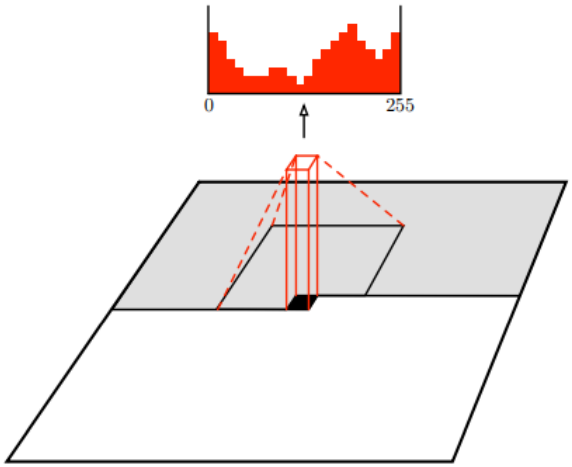
PixelCNN

$$\underbrace{p(\boldsymbol{x})}_{\text{likelihood of input } \boldsymbol{x}} = \underbrace{\prod_{i=1}^{n} p(x_i \mid x_1, \ldots, x_{i-1})}_{\text{probability of } i\text{-th feature given all previous features}}$$

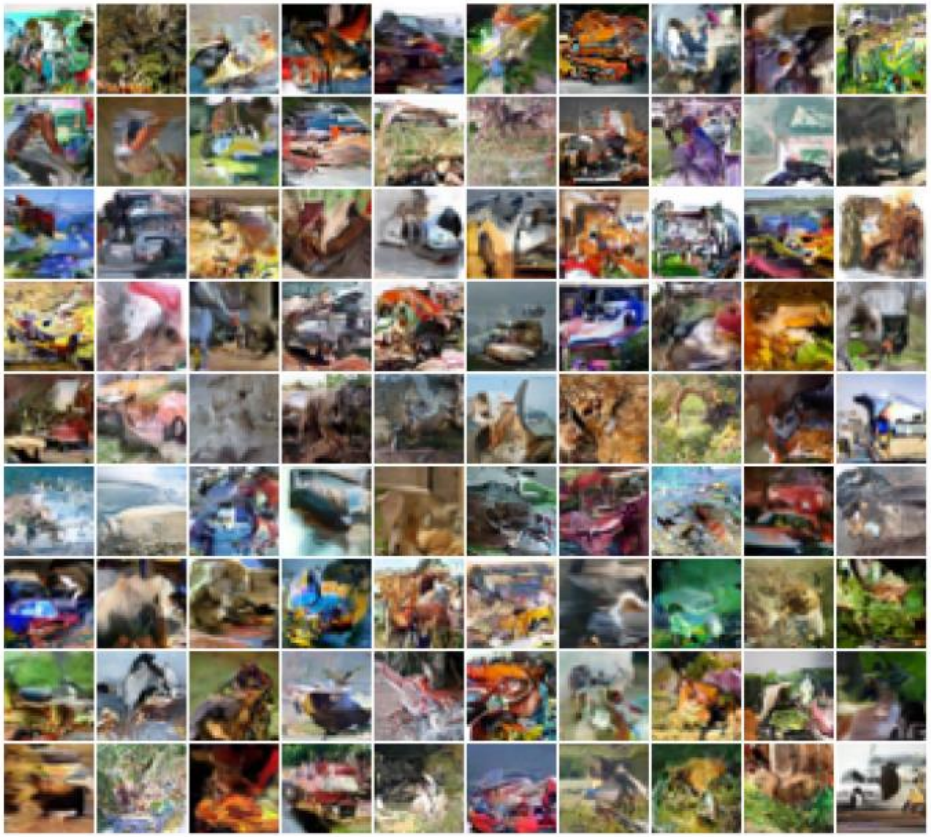
- Still generate image pixels starting from left-top corner
- What's new
  - Use CNN (not LSTM) to model dependency on previous pixels
- training
  - Maximize the likelihood of training images
  - Loss: softmax loss at each pixel
  - Faster than PixelRNN
    - Can parallelize convolutions
- Image generation
  - Must still proceed sequentially

  ⇒ Still low

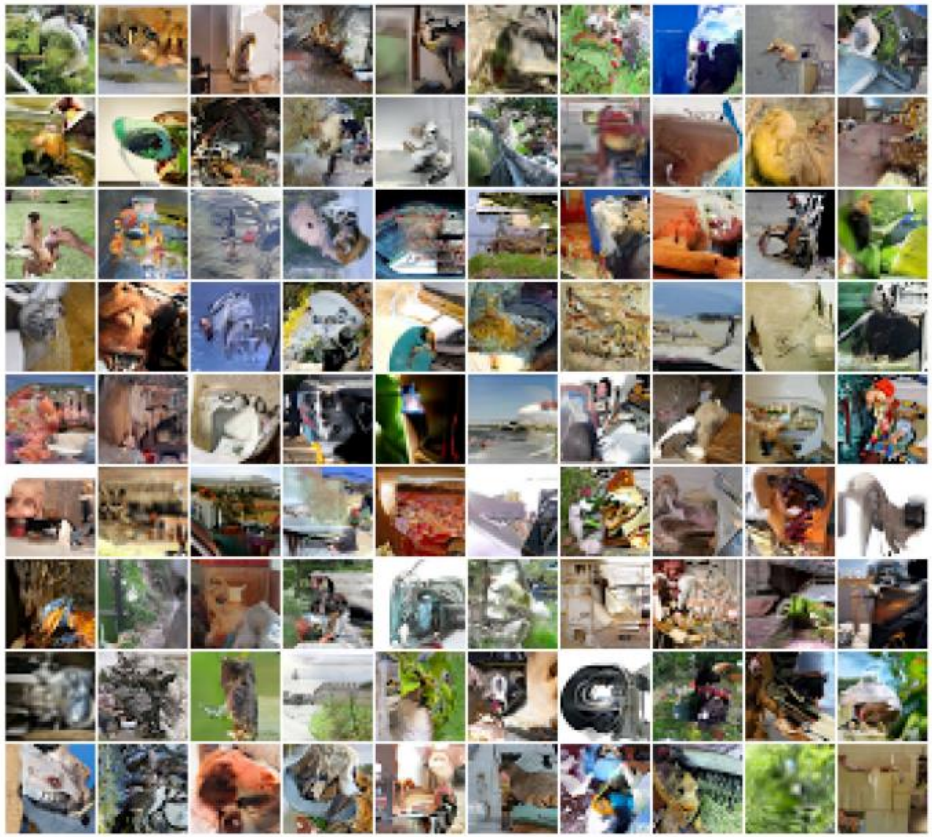
Softmax loss at each pixel



Examples of generated samples



32×32 CIFAR-10



32×32 ImageNet

Remarks on Autoregressive models

- Pros:
  - Can explicitly compute the likelihood
  - Explicit likelihood of training data gives good evaluation metric
  - Good samples
- Cons:
  - Sequential generation ⇒ slow

- Improving PixelCNN performance
  - Gated convolutional layers
  - Short-cut connections
  - Discretized logistic loss
  - Multi-scale
  - Training tricks
  - Etc…

Van der Oord et al. NIPS 2016
Salimans et al. 2017 (PixelCNN++)